Lesson 16 - Homework

Type Hints və Data Classes

Mövzular: Type Hints, funksiyalarda tip işarələri, List/Dictionary/Optional Types, `@dataclass`, Default Values, `field()`, `__post_init__()` və `asdict()`.

Bu ev tapşırığında əvvəlcə mövzuları qısa nümunələrlə təkrarlayacaq, sonra kitabxana idarəetmə sistemini mərhələli şəkildə quracaqsan.

---

1. Xatırlatma (Reminders)

Tapşırıqlara başlamazdan əvvəl dərsdə keçdiyimiz əsas anlayışları yada salaq. Aşağıdakı izahlar və nümunələr tapşırıqları yerinə yetirərkən sizə kömək edəcək.

1.1. Type Hints

Type Hints dəyişənlərin, funksiya parametrlərinin və qaytarılan nəticələrin hansı məlumat tipində olmasının gözlənildiyini göstərmək üçün istifadə olunur.

Əsas məlumat tipləri:

- `str` – Mətn
- `int` – Tam ədəd
- `float` – Onluq ədəd
- `bool` – True və ya False
- `None` – Dəyərin olmaması

Nümunə:

```
ad: str = "Qələm"
qiymet: float = 2.50
stok: int = 15
aktiv: bool = True
```

Yadda saxlayın: Type Hints məlumatların tipini avtomatik yoxlamır. Əsas məqsədi kodu daha anlaşıqlı etmək və mümkün səhvləri əvvəlcədən müəyyənləşdirməyə kömək etməkdir.

1.2. Funksiyalarda Type Hints

Funksiya parametrlərinin tipini `:` işarəsi ilə, qaytarılan nəticənin tipini isə `->` işarəsi ilə göstəririk.

```
def qiymet_hesabla(qiymet: float, say: int) -> float:
    return qiymet * say
```

- `qiymet: float` – Qiymət onluq ədəd olmalıdır.
- `say: int` – Say tam ədəd olmalıdır.
- `-> float` – Funksiyanın onluq ədəd qaytarması gözlənilir.
- `-> None` – Funksiya nəticə qaytarmaq üçün nəzərdə tutulmayıb.

1.3. List, Dictionary və Optional Types

List və dictionary daxilində saxlanılması gözlənilən məlumatların tiplərini də göstərə bilərik.

```
adlar: list[str] = ["Qələm", "Dəftər"]
qiymetler: dict[str, float] = {
    "Qələm": 2.50,
    "Dəftər": 4.00
}
```

- `list[str]` – Mətnlərdən ibarət siyahı.
- `list[int]` – Tam ədədlərdən ibarət siyahı.
- `dict[str, float]` – Key mətn, Value onluq ədəd olan dictionary.

Bəzən dəyişənin dəyəri olmaya bilər. Belə hallarda | `None` yazılışından istifadə edirik.

```
def mehsul_tap(ad: str) -> str | None:
    if ad == "Qələm":
        return "Qələm"
    return None
```

Bu funksiya ya `str`, ya da `None` qaytara bilər.

1.4. Data Classes (`@dataclass`)

Data Class məlumat saxlamaq üçün istifadə etdiyimiz class-ları daha az kodla yaratmağa imkan verir.

```
from dataclasses import dataclass

@dataclass
class Mehsul:
    ad: str
    qiymet: float
    stok: int

qelem = Mehsul("Qələm", 2.50, 15)
print(qelem)
```

`@dataclass` avtomatik olaraq:

- `__init__()` – Constructor yaradır.
- `__repr__()` – Obyektin oxunaqlı göstərilməsini təmin edir.
- `__eq__()` – Obyektləri sahələrinin dəyərlərinə görə müqayisə etməyə imkan verir.

Qeyd: Data Class daxilində adi class-larda olduğu kimi metodlar da yarada bilərik.

1.5. Default Values və `field()`

Data Class sahələrinə standart dəyərlər təyin edə bilərik.

```
@dataclass
class Mehsul:
    ad: str
    qiymet: float
    stok: int = 0
```

Burada `stok` göndərilməzsə, avtomatik `0` qəbul edilir.

List kimi dəyişdirilə bilən məlumat tipləri üçün `field(default_factory=list)` istifadə edirik.

```
from dataclasses import dataclass, field

@dataclass
class Sebet:
    musteri: str
    mehsullar: list[str] = field(default_factory=list)
```

Bu üsul hər yeni səbət üçün ayrıca boş list yaradır.

Yadda saxlayın: Default dəyəri olmayan sahələr default dəyəri olan sahələrdən əvvəl yazılmalıdır.

1.6. `__post_init__()` və məlumatların yoxlanması

`__post_init__()` Data Class obyekti yaradılarkən avtomatik constructor-dan (`__init__` metodu) sonra işləyən metoddur.

Məlumatların müəyyən şərtlərə uyğunluğunu yoxlamaq üçün istifadə edə bilərik.

```
@dataclass
class Mehsul:
    ad: str
    qiymet: float

    def __post_init__(self) -> None:
        if self.qiymet < 0:
            raise ValueError("Qiymət mənfi ola bilməz")
```

Burada qiymət mənfi olarsa, `ValueError` xətası yaranacaq.

Qeyd: `__post_init__()` obyekt yaradılarkən işləyir. Obyektin atributları sonradan dəyişdirildikdə avtomatik təkrar çağırılmır.

1.7. `asdict()` – Data Class-ı Dictionary-yə çevirmək

`asdict()` funksiyası Data Class obyektini dictionary formasına çevirməyə imkan verir.

```
from dataclasses import asdict

mehsul = Mehsul("Qələm", 2.50)

melumat = asdict(mehsul)

print(melumat)
# {'ad': 'Qələm', 'qiymet': 2.5}
```

Bu, məlumatlarla işləyərkən, xüsusilə gələcək backend layihələrində faydalı olacaq.

Tapşırıqlara başlamazdan əvvəl

- Class və object anlayışlarını əvvəlki OOP dərslərindən təkrarlayın.
- Funksiya və metodlarda Type Hints istifadə etməyə çalışın.
- Data Class yaradarkən `@dataclass` əlavə etməyi unutmayın.
- List sahələri üçün `field(default_factory=list)` istifadə edin.
- Məlumatların düzgünlüyünü yoxlamaq tələb olunursa, uyğun şərtlər və `raise ValueError` istifadə edin.
- Kodunuzu yazdıqdan sonra müxtəlif dəyərlərlə yoxlayın. Yalnız düzgün məlumatlarla deyil, yanlış məlumatlarla da test edin.

Vacib: Tapşırıqlarda məqsəd sadəcə işləyən kod yazmaq deyil. Yazdığınız kodun necə və niyə işlədiyini başa düşməyiniz də vacibdir.

2. Ev Tapşırıqları

Mövzu: Type Hints, Data Classes, Default Values, `field()`, `__post_init__()` və `asdict()`

Fayl adı: `lesson_16_hw.py`

Bu dəfə tapşırıqları kiçik bir **kitabxana idarəetmə sistemi** üzərində quracağıq. Hər tapşırıq əvvəlkinin davamı olacaq. Sonda kitabları, oxucuları və kitabxananı idarə edə bilən sadə bir Python proqramımız olacaq.

Bütün tapşırıqları eyni Python faylında yerinə yetirin.

Tapşırıq 1 – Funksiyalara Type Hints əlavə edin

Aşağıdakı funksiyalar artıq yazılıb, lakin Type Hints istifadə olunmayıb.

Sizin işiniz:

1. Hər funksiyanın parametrlərinə uyğun Type Hints əlavə edin.
2. Funksiyaların qaytardığı nəticələrin tiplərini `->` vasitəsilə göstərin.
3. Funksiyaların işləmə məntiqini dəyişməyin.

```
# Funksiya 1: Kitabın gec qaytarılmasına görə cərimə

def cerime_hesabla(gun, gunluk_cerime):
    return gun * gunluk_cerime


# Funksiya 2: Kitabların adlarını göstərmək

def kitab_adlarini_goster(adlar):
    for ad in adlar:
        print(ad)


# Funksiya 3: Kitabların ümumi səhifə sayını hesablamaq

def sehife_cemi(kitablar):
    return sum(kitablar.values())
```

Yoxlama üçün:

```
print(cerime_hesabla(5, 0.50))

kitab_adlarini_goster(["1984", "Alkimyagər", "Səfillər"])

print(sehife_cemi({
    "1984": 328,
    "Alkimyagər": 208
}))
```

Tapşırıq 2 – `Kitab` Data Class yaradın

Kitabxanamızdakı kitabların məlumatlarını saxlamaq üçün `Kitab` adlı Data Class yaradın.

Sahələr:

Sahə	Tip	İzah
`ad`	`str`	Kitabın adı
`muellif`	`str`	Müəllifin adı
`sehife_sayi`	`int`	Səhifə sayı
`qiymet`	`float`	Kitabın qiyməti
`janr`	`str`	Standart dəyər: `"Ümumi"`

Tələblər:

1. `@dataclass` istifadə edin.
2. `janr` üçün default value təyin edin.
3. `__post_init__()` metodu əlavə edin.
4. Səhifə sayı 0 və ya mənfi olarsa, `ValueError` yaradın.
5. Qiymət mənfi olarsa, `ValueError` yaradın.
6. `melumat()` adlı metod yaradın. Kitabın adını, müəllifini və qiymətini mətn şəklində qaytarsın.

Yoxlama üçün:

```
kitab1 = Kitab("1984", "George Orwell", 328, 12.50)
kitab2 = Kitab("Alkimyagər", "Paulo Coelho", 208, 10.00, "Roman")

print(kitab1)
print(kitab2.melumat())
```

Yanlış məlumatı da yoxlayın:

```
# Bu nümunə ValueError yaratmalıdır.
# Yoxladıqdan sonra comment-ə alın.

yanlis_kitab = Kitab("Test", "Müəllif", -100, 5.00)
```

Tapşırıq 3 – `Oxucu` Data Class və kitab götürmə

İndi kitabxanamıza oxucular əlavə edəcəyik.

`Oxucu` adlı Data Class yaradın.

Sahələr:

- `ad` – Oxucunun adı.
- `goturduyu_kitablar` – Oxucunun götürdüyü kitabların siyahısı(list).

Tələblər:

1. `goturduyu_kitablar` üçün `field(default_factory=list)` istifadə edin.
2. `kitab_gotur(kitab)` adlı metod yaradın. Göndərilən kitabı oxucunun siyahısına əlavə etsin.
3. `kitab_sayi()` adlı metod yaradın. Oxucunun götürdüyü kitabların sayını qaytarsın.
4. Bütün metodların parametr və nəticə tiplərini göstərin.

Yoxlama üçün:

```
oxucu1 = Oxucu("Ayxan")
oxucu2 = Oxucu("Tural")

oxucu1.kitab_gotur(kitab1)
oxucu1.kitab_gotur(kitab2)

print(oxucu1.kitab_sayi())  # 2
print(oxucu2.kitab_sayi())  # 0
```

Vacib: İki fərqli oxucu yaratdığınız zaman onların kitab siyahıları bir-birindən asılı olmamalıdır. Bir oxucuya kitab əlavə edildikdə digərinin siyahısı dəyişməməlidir.

Tapşırıq 4 – `Kitabxana` Data Class və kitab axtarışı

İndi kitabları bir yerdə saxlamaq və axtarmaq üçün `Kitabxana` adlı Data Class yaradın.

Sahələr:

- `ad: str` – Kitabxananın adı.
- `kitablar: list[Kitab]` – Kitabxanadakı kitabların siyahısı.

Tələblər:

1. Kitab siyahısı üçün `field(default_factory=list)` istifadə edin.
2. `kitab_elave_et(kitab)` metodu yaradın. Kitabı siyahıya əlavə etsin.
3. `kitab_tap(ad)` metodu yaradın:
    - Kitabın adını parametr kimi qəbul etsin.
    - Siyahı daxilində uyğun kitabı tapsın.
    - Kitab tapılarsa, `Kitab` obyektini qaytarsın.
    - Kitab tapılmazsa, `None` qaytarsın.
4. `kitab_sayi()` metodu yaradın. Kitabxanadakı kitabların sayını qaytarsın.

Yoxlama üçün:

```
kitabxana = Kitabxana("HighCode Kitabxanası")

kitabxana.kitab_elave_et(kitab1)
kitabxana.kitab_elave_et(kitab2)

print("Kitab sayı:", kitabxana.kitab_sayi())

netice = kitabxana.kitab_tap("1984")

if netice is not None:
    print("Tapıldı:", netice.melumat())
else:
    print("Kitab tapılmadı")
```

Eyni axtarışı `"Harry Potter"` adı ilə də yoxlayın.

İpucu:

- `kitab_tap()` metodunun nəticə tipi `Kitab | None` olmalıdır.

Tapşırıq 5 – `asdict()` ilə məlumatların çevrilməsi

İndiyə qədər `Kitab`, `Oxucu` və `Kitabxana` obyektləri yaratdıq.

İndi həmin obyektləri dictionary formasına çevirin.

Tələblər:

1. `dataclasses` modulundan `asdict` import edin.
2. `kitab1` obyektini dictionary-yə çevirin.
3. `oxucu1` obyektini dictionary-yə çevirin.
4. `kitabxana` obyektini dictionary-yə çevirin.
5. Nəticələri `print()` ilə göstərin.
6. `type()` istifadə edərək çevrilmiş məlumatlardan birinin tipini yoxlayın.

İpucu:

```
from dataclasses import asdict

# Obyekti dictionary-yə çevirmək:
asdict(obyekt)
```

`Oxucu` obyektini çevirdikdə onun daxilindəki `Kitab` obyektlərinin necə göstərildiyinə diqqət edin.

Final Tapşırıq – Kitabın geri qaytarılması

Kitabxana sistemimizi bir az da inkişaf etdirək.

Hazırda oxucular kitab götürə bilirlər, amma onları geri qaytarmaq üçün metodumuz yoxdur.

Sizin işiniz:

`Oxucu` Data Class-ına `kitab_qaytar(ad)` adlı yeni metod əlavə edin.

Metod aşağıdakı qaydada işləməlidir:

1. Kitabın adını parametr kimi qəbul etsin.
2. Oxucunun götürdüyü kitabların siyahısını yoxlasın.
3. Kitab tapılarsa, həmin kitabı siyahıdan silsin və `True` qaytarsın.
4. Kitab tapılmazsa, `False` qaytarsın.
5. Parametr və qaytarılan nəticə üçün Type Hints istifadə edin.

Yoxlama üçün:

```
oxucu = Oxucu("Ayxan")

oxucu.kitab_gotur(kitab1)
oxucu.kitab_gotur(kitab2)

print("əvvəl:", oxucu.kitab_sayi())

netice = oxucu.kitab_qaytar("1984")

print("Qaytarıldı:", netice)
print("Sonra:", oxucu.kitab_sayi())

netice = oxucu.kitab_qaytar("Harry Potter")

print("Tapılmayan kitab:", netice)
```

Gözlənilən nəticə:

```
əvvəl: 2
Qaytarıldı: True
Sonra: 1
Tapılmayan kitab: False
```

İpucu:

- `for` dövrü ilə kitabları yoxlayın.
- Uyğun kitabı tapanda `remove()` ilə siyahıdan silə bilərsiniz.
- Kitab silindikdən sonra `return True` istifadə edin.
- Dövr bitənə qədər kitab tapılmasa, `return False` qaytarın.

Qeyd: Tapşırıqları ardıcıllıqla yerinə yetirin. Əvvəlcə işləyən sadə versiyanı yazın, daha sonra növbəti funksionallıqları əlavə edin. Məqsəd kodu əzbərləmək deyil, Type Hints və Data Classes-in real proqramda necə istifadə olunduğunu başa düşməkdir.